Problem Statement

Given an integer array nums and an integer k, return *the* k *most frequent elements*. You may return the answer in any order.

Example 1:

Input: nums = [1,1,1,2,2,3], k = 2

Output: [1,2]

Example 2:

Input: nums = [1], k = 1

Output: [1]

Example 3:

Input: nums = [1,2,1,2,1,2,3,1,3,2], k = 2

Output: [1,2]

Constraints:

- $1 <= nums.length <= 10^5$
- $-10^4 <= nums[i] <= 10^4$
- k is in the range [1, the number of unique elements in the array].
- It is guaranteed that the answer is unique.

Follow up: Your algorithm's time complexity must be better than O(n log n), where n is the array's size.

Program:

```
import java.util.*;


public class Solution {
  public int[] topKFrequent(int[] nums, int k) {
    // Step 1: Count frequencies using HashMap
    Map<Integer, Integer> freqMap = new HashMap<>();
    for (int num : nums) {
      freqMap.put(num, freqMap.getOrDefault(num, 0) + 1);
    }
```

```
    // Step 2: Use a min-heap (PriorityQueue) to keep top k frequent elements
    PriorityQueue<Map.Entry<Integer, Integer>> minHeap =
        new PriorityQueue<>((a, b) -> a.getValue() - b.getValue());

    for (Map.Entry<Integer, Integer> entry : freqMap.entrySet()) {
      minHeap.offer(entry);
      if (minHeap.size() > k) {
        minHeap.poll(); // remove least frequent
      }
    }

    // Step 3: Extract elements from heap
    int[] result = new int[k];
    int index = 0;
    while (!minHeap.isEmpty()) {
      result[index++] = minHeap.poll().getKey();
    }

    return result;
  }
}
```

Sample Output:

Input: nums = [1,1,1,2,2,3], k = 2

Output: [1,2]

Input: nums = [1], k = 1

Output: [1]

Input: nums = [1,2,1,2,1,2,3,1,3,2], k = 2

Output: [1,2]